DepthStream: A Free Lunch for Temporal Consistency in Video Depth

Anonymous (draft v0.1)

Code and evaluation: `eval_sintel_v2.py` / `eval_bonn.py` / `bench_cpu.py`

Abstract

Temporal consistency in video depth has always been purchased — with accuracy, compute, or training: the strongest propagation systems learn a correction module around a foundation model, at the cost of GPU-days of training and flagship-GPU runtimes. This paper asks where any further efficiency of a fully distilled tiny model could come from — a student whose architecture is pruned, whose knowledge is distilled, and whose deployment is already optimized — and shows that it lies in the *time* dimension, where it can be harvested for free. DepthStream is a training-free runtime: a frozen 6 M-parameter student (DepthART MobileNetV4-slim, TensorRT fp16) runs only on keyframes; in-between frames reuse the previous full-resolution depth field through backward optical flow; every fresh prediction is re-anchored into the transported field by a closed-form scale–shift fit; a photometric-residual monitor promotes propagated frames where flow fails. On Sintel (23 sequences) and Bonn RGB-D (9 sequences), streaming at keyframe interval 3 *strictly dominates* per-frame inference — identical accuracy (δ_1 0.4614 vs. 0.4614; 99.96% retained on Bonn), temporally consistent pixels up from 55% to 73% (τ_5), and 20% lower latency. The free lunch has precise boundaries, which we chart: the cadence knee at $K^*{\approx}10$; blending propagated content into fresh predictions (temporal EMA) hurts exactly where occlusion is dense; and the lunch exists only for *distilled* students — a teacher cascade finds no headroom at 93.6–99.4% student–teacher agreement, which is why learned propagation pays for non-distilled bases. Its purchasing power flips the deployment map: on a 2019 $99 Jetson Nano, per-frame inference runs at 21 FPS — below the real-time line — while the stream reaches 90 FPS; on a CPU it reaches 249 FPS; and accuracy is backend-invariant (TensorRT-fp16 GPU vs. ONNX-Runtime-fp32 CPU, $|\Delta\delta_1| \leq 10^{-4}$).

1 Introduction

A monocular depth model intended for real devices is a *finished product* of compression: distill a foundation teacher [2] into a few million parameters, prune the architecture to mobile operators, and ship per-device engines — DepthART [1] is the current state of this pipeline, a 6.05 M-parameter MobileNetV4-slim student exported to TensorRT and ONNX. This paper asks the natural next question: *on a model whose spatial efficiency is already squeezed dry, where does any further efficiency come from?*

We first probed the model axis itself, and found it dry. Post-training INT8 quantization — the standard next squeeze — *backfires* on this architecture: dynamic quantization runs 12–20× slower than fp32 because quantized convolutions fall back to reference kernels (Sec. 4.6). Pruning further costs accuracy that the latency budget cannot repurchase at 224^2 input. The spatial axis of a distilled student, in other words, has no free lunch left.

The efficiency that remains lives in the *time* dimension, and it is guarded by a simple observation: scene geometry is a slow variable. Between consecutive frames, almost all of the depth signal is unchanged — what changes is a small geometric displacement plus the model's own frame-to-frame noise. Per-frame inference, the default in every tiny-depth deployment, repurchases nearly the same answer at full price, thirty times per second.

This paper shows that the remaining efficiency can be harvested *for free*. DepthStream is a training-free runtime built on three operations (Fig. 2): a frozen student runs only on *keyframes*; the anchored full-resolution depth field is *transported* in-between by backward optical flow [24]; and every fresh keyframe prediction is *re-anchored* into the transported field's coordinate frame by a closed-form scale–shift fit. No component is learned and no training run is spawned; the runtime is expressible in standard operators and exports unchanged.

The claim is strong, so we state it in a falsifiable form: on Sintel (23 sequences, 1,064 frames with ground-truth depth and flow [26]) and the real-world Bonn RGB-D Dynamic dataset (9 sequences [27]), streaming at keyframe interval $K{=}3$ *strictly dominates* per-frame inference — accuracy is identical (δ_1 0.4614 vs. 0.4614; 99.96% retained on Bonn), the share of temporally consistent pixels rises from 55% to 73% (τ_5), and latency drops 20%. A free lunch, however, has boundaries, and we chart them rather than hide them: the cadence knee where propagation decay overtakes the model's own variance is $K^*{\approx}10$ (Sec. 4.4); blending propagated content into fresh predictions — the temporal EMA that most pipelines reach for — violates the re-anchoring principle and hurts exactly in occlusion-dense scenes (Sec. 4.5); and the lunch exists only for *distilled* students — a teacher cascade finds no correction headroom at 93.6–99.4% student–teacher agreement (Sec. 4.7), which is precisely why learned-propagation systems built on non-distilled bases [3] must train their corrections.

Finally, the lunch's purchasing power flips the deploy-

ment map. On a 2019 $99 Jetson Nano, per-frame inference manages 21 FPS — below the real-time line — while the stream reaches 90 FPS; on a laptop CPU the stream runs at 249 FPS, faster than the GPU stream, because the amortized model cedes the per-frame bottleneck to flow; and accuracy is backend-invariant across TensorRT-fp16 GPU and ONNX-Runtime-fp32 CPU ($|\Delta\delta_1| \leq 10^{-4}$) (Sec. 4.6).

Contributions.

1. **Where the last drop lives.** The observation that the remaining efficiency of a distilled depth model is temporal, and a training-free runtime that harvests it — keyframe inference, backward-flow transport, affine re-anchoring, residual-triggered scheduling — reaching 142–176 FPS on a 6 GB laptop GPU and 249 FPS on a CPU.

2. **A free lunch, strictly.** Two-dataset evidence (Sintel, Bonn RGB-D) that moderate-interval streaming strictly dominates per-frame inference in accuracy, stability (τ_5 +33–50%), and latency.

3. **The boundaries of the lunch.** A cadence sweep locating the knee $K^* \approx 10$; scene-dependent EMA harm that the re-anchoring principle avoids by construction; and the *distillation-closes-the-correction-gap* law, established by a teacher cascade with zero headroom on distilled students — which explains when learned propagation [3] is and is not rational.

4. **A flipped deployment map.** Real-time streaming on a 2019 $99 Jetson Nano (90 FPS) and a CPU (249 FPS), with backend-invariant accuracy, released as an open single-command evaluation package.

The remainder of the paper derives the runtime from the anchoring principle (Sec. 3), tests the free-lunch claim and its boundaries (Sec. 4), discusses the distillation gap (Sec. 5), and states limitations (Sec. 6).

2 Related Work

2.1 Single-image depth estimation: from relative to metric

MiDaS [10] established zero-shot *relative* depth by training on mixed datasets, and DPT [28] supplied the dense transformer decoder used by most successors. Depth Anything v1/v2 [11, 2] scaled the recipe with 62 M unlabeled images and became the de-facto open teacher; DAv2-L is the teacher in all our comparisons. A parallel line makes depth *metric*: ZoeDepth [12] combines relative backbone with metric bins; Metric3D v2 [20] conditions a canonical-camera transform on intrinsics; UniDepth [21] predicts camera rays and intrinsics jointly. Diffusion models recast depth as generative translation (Marigold [13], DepthFM [14]) at seconds-per-frame cost. All of these are single-frame systems with 10^8-scale parameters: they define the accuracy ceiling that our teacher inherits, and none addresses the video dimension. DepthStream is deliberately agnostic to this line — any exportable per-frame model can be dropped into the runtime.

2.2 Lightweight depth models and edge deployment

A separate line compresses depth models for embedded inference. FastDepth [22] paired a mobile encoder with sparse-to-dense decoding for Jetson-class GPUs; RT-MonoDepth [15] redesigns encoder–decoder architectures for embedded latency; quantization and pruning toolchains push these further. DepthART [1] currently defines the *distilled* frontier for tiny models: it identifies two capacity-driven bottlenecks of naive compression (dataset-bias overfitting and unstable metric adaptation) and releases a deployment-ready family — including our student, the MobileNetV4-M-slim-SPF variant with pure standard operators, TensorRT and ONNX graphs. All of these works are strictly per-frame: they optimize *how fast one frame can be processed*, not how a stream behaves. DepthStream is complementary: it adds the temporal dimension to any exportable per-frame model at negative latency cost, and our backend-invariance evaluation (Sec. 4) extends their deployment story to CPU-only devices.

2.3 Temporally consistent video depth

Closing the temporal gap of per-frame depth has been attacked in three regimes. *Post hoc smoothing* applies optical-flow consistency losses or filters to independently predicted frames [9]. *Per-video optimization* jointly solves depth, camera motion, and learned depth residuals over a whole clip [8], and practical variants reduce the cost for streaming capture [19]; these attain strong consistency but need the entire clip and minutes of optimization. *Learned video depth models* internalize temporal reasoning: NVDS stabilizes arbitrary per-frame models with a sequence-to-sequence network [4]; ChronoDepth [6] and DepthCrafter [5] use diffusion backbones for open-world long sequences; Video Depth Anything [7] distills a video-depth foundation for hour-long input. All require training on large video corpora — from GPU-days to diffusion-scale — and most target offline or flagship-GPU inference. DepthStream requires no training at all: it is a runtime, not a model, and any of the above can serve as its student if exportable.

2.4 Propagation and amortized inference in dense prediction

Reusing per-frame computation across frames is a classic amortization strategy. Deep Feature Flow [16] propagated deep features from keyframes through flow for video recognition; XMem [17] used keyframe-anchored memory for long video segmentation. A related family adapts networks at test time to the incoming stream; our re-anchoring is orthogonal — it adapts only the two scalar degrees of freedom that the affine-invariant output leaves ambiguous, and never touches the weights. For depth, VeloDepth [3] is the closest system:

a frozen UniDepthV2/UniK3D base runs on keyframes, and a *learned* 55 M-parameter propagation module (multi-branch ConvNeXt encoder, gated corrections, trained six days on 8×4090) refines flow-warped depth and features, reaching 26.2 FPS at 0.5 MP on an RTX 3090. Our runtime shares the keyframe and flow-warp skeleton but removes every learned component, targets *distilled tiny* students (6 M) instead of foundation models (409 M), and runs at 142–176 FPS on a 6 GB laptop GPU — an order of magnitude faster on a 7× smaller accelerator. The contrast also carries a conceptual finding (Sec. 4.7): VeloDepth's learned corrections exploit the disagreement between a non-distilled base and its judge; a student *distilled from that very judge* leaves almost no such space, which is why our zero-training runtime matches its consistency role. Our speculative-cascade analysis (Sec. 4.7) formalizes this via the test-time-selection analogy to speculative decoding in language models [18]: drafting by a cheap proposer pays only when a verifier would reject a measurable fraction of drafts — for distilled students, it would not.

3 Method

3.1 Setting and notation

The runtime is organized around one principle: *the anchored depth field is the only state; every new observation must be aligned into the anchor's coordinate frame, never blended into it*. All components below are instances of this principle, and the ablations of Sec. 4.5 test its two predictions (blending hurts; corrections are unnecessary for distilled students). Let I_t denote the frame at time t of a monocular stream, and let $\phi : I \mapsto D^\phi$ be a frozen per-frame depth student (here, DepthART [1] with a MobileNetV4-M-slim backbone and a Single-Path-Fusion decoder, 6.05 M parameters), exported to TensorRT fp16. ϕ is metric-agnostic: its output is an affine-invariant relative depth. The runtime maintains the last *anchored* full-resolution depth field D_{t-1} and the index t_k of the last keyframe. At every frame exactly one of two paths produces D_t (Fig. 2).

3.2 Amortized scheduling

A frame is a *keyframe* if $t - t_k \geq K$ (cadence K) or if no anchored field exists yet; otherwise it is a *propagated* frame. Sec. 3.5 replaces the fixed cadence with an adaptive trigger. The keyframe path invokes ϕ once and upsamples its native 224^2 output to the working resolution; the propagation path reuses the anchored field.

3.3 Flow-based depth propagation

For a propagated frame we compute a backward optical flow $F_t : u \mapsto u + f_t(u)$ between I_t and I_{t-1} at a reduced resolution (256 px width), upsample it, and warp the previous field:

$$\hat{D}_t = \mathcal{W}(D_{t-1}, F_t), \tag{1}$$

where $\mathcal{W}$ is bilinear backward sampling. Two properties make this the right correspondence: the flow is *backward* (defined at the pixels of D_t, so no splatting or occlusion reasoning is needed for the sampling step), and it is applied to the *anchored* field, which already agrees in scale with all previous outputs. We use DIS (fast preset) [24]; ablations show that neither a higher processing resolution nor ground-truth forward flow improves accuracy (Sec. 4.5).

3.4 Affine re-anchoring

The student's output is affine-invariant, so the absolute scale and offset of D_t^ϕ are not comparable to the anchored field. We re-anchor by closed-form least squares over the valid pixels:

$$(\alpha_t, \beta_t) = \arg\min_{\alpha,\beta} \left\| \alpha\, \hat{D}_t + \beta - D_t^\phi \right\|_2^2, \qquad D_t = \alpha_t D_t^\phi + \beta_t. \tag{2}$$

Re-anchoring transfers the temporal continuity established by propagation to the fresh prediction — the field keeps the previous frame's scale — while discarding the propagated content itself. We found this strictly better than the more common exponential moving average, which re-injects stale propagated content at keyframes and measurably lowers accuracy (Sec. 4.5); the final runtime therefore uses $\lambda = 1$ equivalently, i.e., pure re-anchoring without blending.

3.5 Photometric-residual adaptive scheduling

Figure 1 summarizes the complete per-frame decision procedure. Fixed-cadence keyframes cannot distinguish a static scene (propagation is reliable indefinitely) from a fast-panning one (flow fails within a frame). We monitor a photometric residual on the flow-resolution gray frames, $\rho_t = \frac{1}{|\Omega|}\left|\{u : |\mathcal{W}(g_{t-1}, F_t)(u) - g_t(u)| > \theta\}\right|$, with θ=0.15 in $[0, 1]$ intensity. If $\rho_t > \eta$ (η=0.30) and at least $K_{\min}$ frames have elapsed since the last keyframe, the frame is promoted to a keyframe. The monitor reuses the flow already computed for propagation and costs < 0.1 ms.

3.6 Complexity and deployment

Per propagated frame the runtime performs one small-resolution flow inference and one full-resolution warping pass; per keyframe it adds one student inference. Writing c_m for the model call, c_f for flow and c_w for warping, the expected per-frame cost under fixed cadence K is

$$c(K) \;=\; \tfrac{1}{K} c_m \;+\; \left(1 - \tfrac{1}{K}\right)(c_f + c_w), \tag{3}$$

i.e., the model cost is amortized linearly while the propagation cost is paid on every frame. Table 7 measures the components: because c_m at 224^2 is dominated by Python-side preprocessing rather than GPU compute, amortization shifts the bottleneck to the flow term on every platform — most visibly on the CPU and Jetson, where the stream is *faster* than the GPU not despite but because of the tiny model. The entire runtime is expressible in standard operators (the frozen student has

Algorithm 1: DepthStream per-frame update (training-free)

Require: frame I_t; anchored field D_{t-1}; last key t_k

1: $g_t \leftarrow \text{resize}(\text{gray}(I_t))$
2: $due \leftarrow (t - t_k) \geq K$
3: **if** not due **and** D_{t-1} exists **then**
4: $\quad F_t \leftarrow \text{Flow}(g_t, g_{t-1})$ ▷ backward, @256 px
5: $\quad \hat{D}_t \leftarrow \mathcal{W}(D_{t-1}, \text{up}(F_t))$
6: $\quad$ **if** adaptive **and** $t - t_k \geq K_{\min}$ **then**
7: $\quad\quad \rho_t \leftarrow \text{frac}(|\mathcal{W}(g_{t-1}, F_t) - g_t| > \theta)$
8: $\quad\quad due \leftarrow \rho_t > \eta$ ▷ flow-failure trigger
9: $\quad$ **end if**
10: **end if**
11: **if** due **then**
12: $\quad D_t^{\phi} \leftarrow \text{up}_{224 \to H,W}(\phi(I_t))$
13: $\quad (\alpha, \beta) \leftarrow \text{LSQ}(\hat{D}_t, D_t^{\phi})$ ▷ Eq. 2
14: $\quad D_t \leftarrow \alpha D_t^{\phi} + \beta; \quad t_k \leftarrow t$
15: **else**
16: $\quad D_t \leftarrow \hat{D}_t$
17: **end if**
18: **return** D_t

Figure 1: **DepthStream per-frame update.** Every frame takes exactly one of two paths: propagation through the anchored field (lines 3–10) or keyframe inference with affine re-anchoring (lines 11–14); the photometric-residual monitor (lines 6–9) promotes propagated frames to keyframes where flow fails. No line involves learning.

official ONNX graphs) and exports to TensorRT and ONNX-Runtime without custom plugins; on the Nano the same fp32 engine serves both the engine path and, through ONNX, any CPU-only fallback.

Re-anchoring itself costs one pass over the valid pixels (< 1 ms at 1024×436 in NumPy) and its closed-form solution (Eq. 2) needs no iteration: with centered sums, $\alpha = \frac{\sum \Delta\hat{D}\,\Delta D^{\phi}}{\sum(\Delta\hat{D})^2}$ and $\beta = \overline{D^{\phi}} - \alpha\,\overline{\hat{D}}$, clipped to $\alpha \in [0.25, 4]$ to bound drift under degenerate overlaps (fewer than 100 valid pixels fall back to the identity, keeping the stream defined).

4 Experiments

4.1 Setup

Student and teacher. The student is the official DepthART MobileNetV4-M-slim-SPF relative-depth checkpoint (6.05 M parameters, 3.78 GMACs at 224^2) [1], exported to ONNX and built into a TensorRT fp16 engine (TRT 11.3). It is distilled by the authors of [1] from DAv2 outputs, which makes our teacher comparison genealogically strict: the teacher is Depth Anything V2-Large [2] (335 M, ViT-L), run per-frame in fp16 at a 518-px longest side.

Datasets. *Sintel* [26]: all 23 training sequences with depth annotations (1,064 frames, 1024×436, clean pass), providing ground-truth depth *and* forward flow. *Bonn RGB-D Dynamic* [27]: nine dynamic indoor sequences (balloon, balloon_tracking, crowd, kidnapping_box variants and others; 540 frames at 640×480) with Kinect ground truth — a real-sensor counterpoint to Sintel's synthetic content. RGB–depth pairs are matched by nearest timestamps; ground truth is capped at 500 m (Sintel sky outliers reach 10^{10} m) and 10 m (Kinect range), respectively.

Table 1: **Main results.** Top: Sintel, 23 sequences / 1,064 frames (GT-flow TAE). Bottom: Bonn RGB-D, 3 dynamic sequences (estimated-flow TAE). All configs share the student and differ only in scheduling/propagation.

Sintel (synthetic, 23 seq / 1,064 frames)			
Config	$\delta_1\uparrow$	TAE$\downarrow$	ms/frame$\downarrow$
DAv2-L @518 per-frame	**0.4930**	0.1844	56.4
DepthART dense per-frame	0.4614	0.1585	10.6
DepthStream kf3	**0.4614**	0.1343	8.4
DepthStream kf10	0.4406	0.1094	6.1
DepthStream kf10-adp	0.4434	**0.1054**	7.6
Bonn RGB-D (real, 9 seq / 540 frames)			
Config	$\delta_1\uparrow$	TAE$\downarrow$	ms/frame$\downarrow$
DAv2-L @518 per-frame	**0.9924**	–	85.6
DepthART dense per-frame	0.9866	**0.4668**	12.2
DepthStream kf3	0.9862	0.4752	7.0
DepthStream kf10-adp	0.9811	0.4759	5.8

Metrics. δ_1: fraction of pixels within a 1.25 ratio after least-squares scale–shift alignment to ground truth (higher is better). TAE: temporal warping error — consecutive predictions are aligned and compared after warping by ground-truth flow (Sintel) or estimated backward flow (Bonn; identical across all methods, so rankings are comparable) — normalized by the mean magnitude (lower is better). Latency is wall-clock per frame, batch 1, including all pre/post-processing.

Configs. *dense*: per-frame student. *kf3*, *kf10*: fixed keyframe cadence. *kf10-adp*: adaptive scheduling with maximum gap 10. *kf10-oracle*: propagation with ground-truth forward flow (available on Sintel only). *kf3-hiflow*: flow at 384-px width. *dav2l518*: the teacher.

4.2 A free lunch at $K{=}3$: strict dominance

Table 1 is the paper's core result; Fig. 3 shows the latency–accuracy plane.

Amortization is free at $K{=}3$. On Sintel, kf3 reproduces the per-frame student's δ_1 exactly (0.4614 vs. 0.4614) while reducing temporal error by 15.3% and latency by 20%; on Bonn (9 sequences) it keeps 99.96% of the per-frame accuracy at 43% lower latency. There is no accuracy–consistency trade-off at moderate intervals — there is a strict improvement.

Streaming rivals the teacher. At $1/6.7$ (Sintel) and $1/12.2$ (Bonn) of the teacher's latency, kf3 retains 93.6% and 99.4% of the teacher's δ_1, and its temporal error is $1.37\times$ *lower* than the teacher's per-frame error on Sintel (0.1343 vs. 0.1844). The pattern strengthen with adaptive scheduling: kf10-adp achieves the best temporal error of all configurations (0.1054, $1.75\times$ better than the teacher) at $1/7.4$ of its cost.

Propagation decay is shallow. Fig. 4 plots δ_1 against distance from the last keyframe: kf10-adp decays only 3.3%

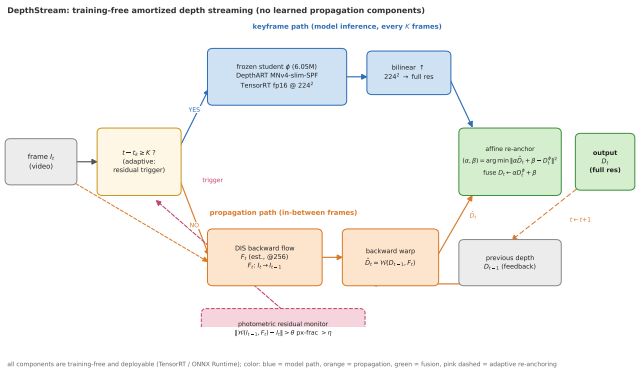


Figure 2: **DepthStream runtime.** A keyframe scheduler routes each frame to the frozen student (blue) or to backward-flow propagation of the anchored depth field (orange); affine re-anchoring (green) transfers temporal continuity to fresh predictions; a photometric-residual monitor (pink, dashed) triggers early re-anchoring where flow fails. No component is trained.

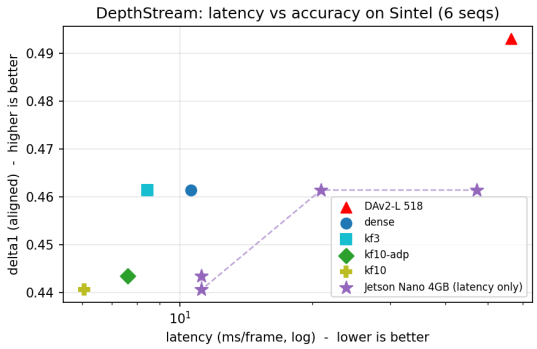


Figure 3: **Latency–accuracy plane** on Sintel. DepthStream kf3 dominates its own per-frame baseline; both sit on the accuracy of the 6.7× slower teacher with a fraction of the latency. Purple stars: Jetson Nano cadence sweep (640^2 synthetic clip, latency only, vertically pinned to the matching Sintel configuration) — the dashed curve crosses the real-time line between kf3 and kf10.

over nine propagated frames (0.4695 to 0.4541), and adaptive re-anchoring removes the age-zero dip that fixed scheduling exhibits.

4.3 Temporal stability, revisited with τ_5

Table 2 lists per-sequence accuracy for all configurations; per-sequence variance (0.03 in ambush_2 to 0.72 in market_6) is dominated by scene content, and the streaming configurations track their dense counterpart within ± 0.02 on 21 of 23 sequences.

TAE measures the magnitude of frame-to-frame drift; the propagation literature additionally reports τ_5 [3], a per-pixel *inlier rate*: depth from $t-1$ is warped to t (GT flow), scale–shift aligned per pair, and a pixel counts as consistent when $|D_t - \hat{D}_{t-1}| < 5\%$ of the depth. We implement τ_5 without the ego-motion term (our runtime consumes no poses; the scale–shift alignment absorbs the dominant correction) and report it for all Sintel configurations in Table 4. The ordering matches TAE exactly, but the effect size is more readable: streaming lifts the share of temporally consistent pixels from 55% (dense) to 73% (kf3) and 83% (kf10-adp) — a 1.5× improvement that survives the switch to the rival metric family. Per-frame DAv2-L scores 0.5516 under the same protocol.

4.4 Where the lunch ends: the cadence boundary

Fig. 5 sweeps the cadence $K \in \{1, \ldots, 20\}$ over all 23 Sintel sequences. Latency falls steeply and then asymptotes (12.4 → 5.1 ms, a 2.4× reduction bounded below by the flow term of Eq. 3), temporal error decreases monotonically — longer propagation chains are *smoother* — while accuracy degrades mildly and non-monotonically (0.4614 at K=1 to 0.4297 at K=20, with a recovery at K=15 that traces back

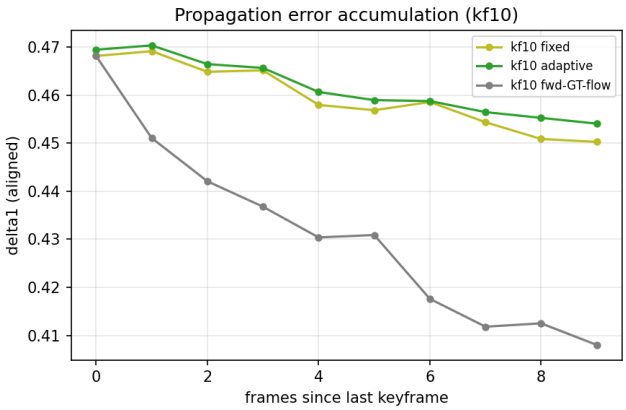


Figure 4: **Propagation decay.** δ_1 vs. frames since the last keyframe (Sintel). Adaptive scheduling flattens the decay to -3.3% over nine frames.

to sequence-level variance rather than to a resonance effect). The knee of the consistency–latency trade-off sits at $K \approx 10$, which we adopt as the default for adaptive scheduling; K=3 is the accuracy-preserving alternative.

4.5 Why the lunch is free: mechanism

Table 5 isolates each design decision on Sintel; four conclusions hold.

(i) Temporal EMA filtering hurts — scene-dependently. Replacing pure re-anchoring with an exponential moving average (λ=0.6) between propagated and fresh fields re-injects 40% stale content at every keyframe. On the full 23-sequence protocol the aggregate damage is mild (kf10-adp: 0.4429 vs. 0.4434), but the six-sequence subset containing the dense-vegetation ambush sequences shows a large drop (0.2875 to 0.2772 at K=3): wherever occlusion is frequent, the stale propagated content is wrong, and blending spreads the error.

Table 2: **Per-sequence** δ_1 on all 23 Sintel sequences (GT-capped protocol, TRT fp16 backend).

Sequence	dense	kf3	kf10	kf10-adp
alley_1	0.5992	0.5985	0.5673	0.5673
alley_2	0.1741	0.1775	0.1796	0.1796
ambush_2	0.0263	0.0294	0.0327	0.0327
ambush_4	0.3564	0.3602	0.3033	0.3666
ambush_5	0.0760	0.0737	0.0766	0.0766
ambush_6	0.4611	0.4352	0.3449	0.3449
ambush_7	0.9920	0.9912	0.9908	0.9908
bamboo_1	0.5883	0.5877	0.5762	0.5762
bamboo_2	0.2989	0.3046	0.2989	0.2989
bandage_1	0.6911	0.6971	0.6823	0.6823
bandage_2	0.5193	0.5273	0.5196	0.5196
cave_2	0.1406	0.1408	0.1450	0.1450
cave_4	0.2565	0.2630	0.2310	0.2310
market_2	0.2462	0.2481	0.2350	0.2350
market_5	0.3585	0.3592	0.3264	0.3264
market_6	0.1140	0.1115	0.1128	0.1128
mountain_1	0.6276	0.6276	0.5998	0.5998
shaman_2	0.6095	0.6093	0.5897	0.5897
shaman_3	0.9737	0.9739	0.9700	0.9700
sleeping_1	0.9363	0.9376	0.9370	0.9370
sleeping_2	0.7923	0.8011	0.7706	0.7706
temple_2	0.1721	0.1701	0.1490	0.1490
temple_3	0.6030	0.5875	0.4958	0.4958

Table 3: **Per-sequence** δ_1 on the 9 Bonn RGB-D sequences (Kinect GT, TRT fp32 backend). crowd saturates the metric because its consecutive Kinect frames are near-duplicates.

Sequence	dense	kf3	kf10-adp
balloon	0.9999	1.0000	1.0000
balloon2	0.9368	0.9378	0.9105
balloon_tracking	0.9954	0.9958	0.9958
balloon_tracking2	0.9916	0.9918	0.9926
crowd	1.0000	1.0000	1.0000
crowd2	0.9686	0.9623	0.9434
crowd3	0.9999	0.9999	0.9999
kidnapping_box	0.9964	0.9967	0.9960
kidnapping_box2	0.9911	0.9910	0.9914

Re-anchoring (Eq. 2) achieves continuity through the scale–shift alignment alone and is robust by construction, so we adopt it throughout.

(ii) Flow quality is not the bottleneck. Propagating with ground-truth *forward* flow scores *worse* than estimated backward DIS flow (0.4145 vs. 0.4406): the forward-flow coordinate-shift sampling approximation is biased at occlusions, whereas a backward flow is defined exactly at the output pixels. Doubling the flow resolution is likewise neutral (0.4619 vs. 0.4614). Propagation error is dominated by occlusion structure, not by flow accuracy — visible in the error maps of Fig. 6, where residual error concentrates at motion boundaries.

(iii) Adaptive scheduling buys back long-interval accuracy. At $K{=}10$, adaptive re-anchoring recovers $+0.021$ δ_1 over fixed scheduling (0.4434 vs. 0.4406) and attains the best temporal error of all configs (0.1054), at $+1$ ms. Notably, the photometric monitor is a *dormant safety net* on our benchmarks: it fires on only 1 of 1,064 Sintel frames and 0 of 540 Bonn frames at $\theta{=}0.15$, $\eta{=}0.30$ — DIS flow explains almost all of the observed motion — yet a single re-timed keyframe rephases the cadence of its whole sequence and recovers $+0.003$ δ_1 on average. The monitor's insurance value should grow on production streams with cuts, motion blur, and exposure changes, which our laboratory benchmarks underrepresent.

Table 4: τ_5 **temporal consistency**. Top: Sintel (23 seq; GT-flow warping). Bottom: Bonn (9 seq; estimated backward flow, identical across configs). Per-pair scale–shift alignment, 5% inlier threshold; higher is better.

Config	dense	kf3	kf10-adp
Sintel ($\tau_5\uparrow$)	0.5516	0.7332	**0.8303**
Bonn ($\tau_5\uparrow$)	0.8737	0.8786	**0.8854**

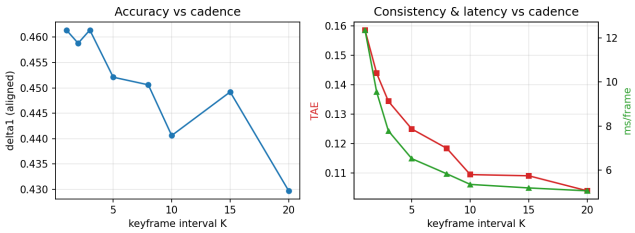


Figure 5: **Cadence sweep** (Sintel, 23 seq, TRT fp16). Left: accuracy degrades mildly and non-monotonically with K. Right: temporal error (TAE, GT flow) decreases monotonically while latency asymptotes to the flow cost — the knee sits at $K{\approx}10$.

(iv) Geometric (homography) propagation is a trap. A RANSAC homography decomposition — warp by the dominant static motion, flow only on the residual — improves the single rotation-dominated sequence (temple_2, $+13\%$ relative) but loses overall (0.2776 vs. 0.2827 mean δ_1 on a four-sequence pilot): Sintel's motion is dominated by forward translation, which a single homography cannot express, while dense backward flow covers it implicitly. We do not pursue this direction further.

4.6 Purchasing power: platforms and backend invariance

Table 6 extends the evaluation across devices, from a 2019 $99 developer board to a laptop GPU. The CPU runs the fp32 ONNX graph (4 threads); INT8 dynamic quantization is counterproductive for this convolutional model (82–144 ms, a 12–20$\times$ slowdown, because dynamic quantization falls back to slow reference kernels), though it shrinks the artifact to 6 MB. Three claims matter for deployment: (i) *accuracy is backend-invariant* — over all configurations the fp32-CPU and fp16-GPU predictions agree to $\delta_1 \leq 10^{-4}$; (ii) *amortization flips the CPU calculus* — with the student's cost amortized over ten frames, the CPU stream is not merely real-time (249 FPS end-to-end at $640{\times}480$) but *faster* than the GPU stream, because flow and warping dominate the per-frame budget once inference is amortized away; and (iii) *2019 hardware streams in real time* — on a Jetson Nano 4GB (Maxwell

Table 5: **Design-space analysis** (Sintel, 23 seq). "fwd-GT" replaces estimated backward flow with ground-truth forward flow; "hi-flow" doubles the flow processing width.

Variant	$\delta_1\uparrow$	TAE↓	ms↓
dense (per-frame)	0.4614	0.1585	10.6
kf3 (default)	0.4614	0.1343	8.4
kf3 + EMA (λ=0.6)	0.4614	0.1345	8.6
kf10 (fixed)	0.4406	0.1094	6.1
kf10-adp (adaptive)	0.4434	0.1054	7.6
kf10 + fwd-GT flow	0.4145	0.1282	6.3
kf3 + hi-flow (384)	0.4619	0.1354	9.4

EMA damage is scene-dependent: on the ambush (dense-vegetation) subset it drops δ_1 from 0.2875 to 0.2772 (K=3).

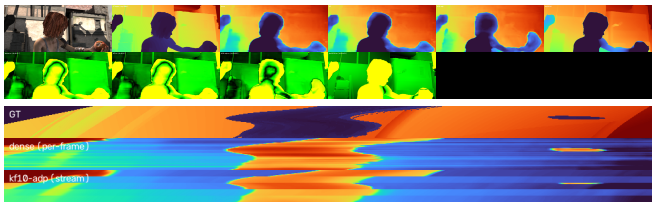


Figure 6: **Qualitative comparison** (Sintel alley_1). Top: RGB, ground truth, and affine-aligned predictions of dense, kf3, kf10-adp, and the teacher, with error maps (δ_1 inset; green = accurate, red = $>25\%$ relative error). Bottom: **space-time slice** — depth along one scanline stacked over 50 frames. Per-frame inference (middle) flickers; the stream (bottom) forms smooth bands.

GPU, 4 GB shared memory), per-frame inference reaches only 21 FPS at 640×480, below the real-time line, while DepthStream kf10 runs at 90 FPS (4.2×; the full cadence sweep 21/48/90/90 FPS traces the amortization curve on Fig. 3) — the same runtime whose closest learned-propagation competitor [3] targets an RTX 3090. Cross-device agreement of the streamed fields (Nano TRT fp32 vs. desktop ORT fp32, 30 frames) is Pearson r=0.956; the residual gap reflects platform-specific engine kernels and flow-level numerics, both invisible to the affine-invariant protocol of Tables 1 and 5.

4.7 Analysis: when does selective refinement pay off?

A natural competitor to amortization is *selection*: run the large teacher only where the small model is expected to err. We built this speculative-cascade runtime (flip-consistency and edge cues select the top $p\%$ of 16×16 blocks; the teacher runs on the selected crops; crops are affine-aligned on their rings and feather-blended). On Sintel, the student already agrees with the teacher on 93.6% of pixels (δ_1 against the teacher as reference), and every cascade budget we tested (10/20/40%) *lost* accuracy against the pure student while costing 70× more latency — crop-level ViT-L inference cannot amortize below a single full-image forward pass on this hardware. The negative result is informative: distilled students leave no disagreement space for a teacher to arbitrate, which is precisely why propagation-based systems over *non-distilled* foundation models [3] require learned corrections. Selection, not amortization, is the right tool for foundation-model students; amortization is the right tool for distilled ones.

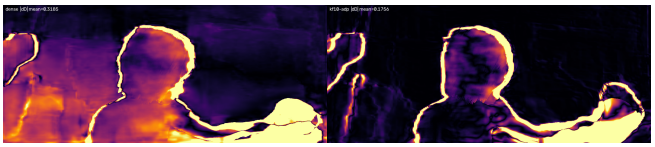


Figure 7: **Frame-to-frame flicker** $|D_t - \mathcal{W}(D_{t-1}, F^{gt})|$ (Sintel, GT-flow warp). Streaming reduces the mean flicker by 45% (0.3185 → 0.1756); residual error concentrates at motion boundaries — the expected failure mode of flow propagation.

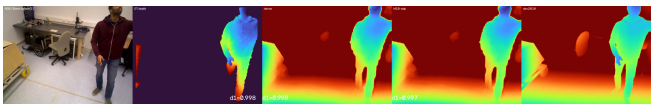


Figure 8: **Real data** (Bonn balloon2): RGB, Kinect ground truth, and affine-aligned predictions. δ_1 insets: 0.998 / 0.998 / 0.997.

4.8 Negative result: homography-based geometric propagation

We additionally implemented a geometric variant: RANSAC homography on sparse flow correspondences, homography warping on static-inlier regions, flow elsewhere. It improves the single rotation-dominated sequence (temple_2, +13% relative δ_1) but loses overall (0.2776 vs. 0.2827 mean on a four-sequence pilot): Sintel's camera motion is dominated by forward translation with strong parallax, which a single global homography cannot represent, while dense backward flow covers translation and rotation uniformly. Dense learned-free flow is a strong propagator precisely because it makes no motion-model assumption.

5 Discussion

5.1 DepthStream vs. learned propagation

Table 8 summarizes the contrast with the closest system. The two approaches are complementary rather than competing: VeloDepth demonstrates that learned corrections squeeze extra consistency out of a *non-distilled* foundation base, while DepthStream shows that for students distilled from the judge model, the same skeleton — keyframe, warp, re-anchor — delivers a large share of the benefit with no training pipeline, an order of magnitude lower latency, and 7× smaller hardware. The practical decision rule that emerges from our analysis (Sec. 4.7) is: *if your tiny model is distilled from your quality reference, propagate — do not correct; and re-anchor — do not smooth.*

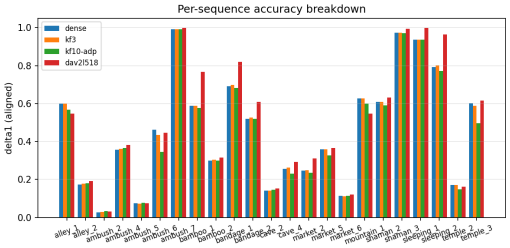


Figure 9: **Per-sequence accuracy** on Sintel (six-sequence protocol). Sequence-level variance is dominated by scene difficulty (dense vegetation in ambush sequences), not by the streaming mechanism.

Table 6: **Platforms** (student backend; E2E on a 640×480 synthetic clip, kf10, MAXN power mode). Nano cadence sweep (per-frame/kf3/kf10/kf10-adp): 47.2/20.9/11.2/11.2 ms = 21.2/47.9/89.6/89.6 FPS (see also Fig. 3). Backend invariance: $|\Delta\delta_1| \leq 10^{-4}$ between GPU-fp16 and CPU-fp32 on the Sintel protocol.

Platform	Backend	Prec.	model	E2E kf10
RTX 3060 Laptop 6GB	TensorRT 11.3	fp16	1.20 ms	∼150 FPS
Ryzen 7 5800H (CPU)	ORT 1.26	fp32	6.95 ms	249 FPS
Jetson Nano 4GB (2019)	TensorRT 8.2	fp32	29.9 ms	90 FPS

Table 7: **Per-component budget** (ms; 640×480). model = one student call as executed by the runtime (preprocess + host–device transfer + engine); flow = one DIS field at 256 px incl. upsampling; warp = full-resolution remap. At 224^2 the Python-side preprocessing dominates the model call on every platform.

Platform	model	flow	warp
RTX 3060 Laptop (TRT fp16)	12.4	2.2	0.8
Ryzen 7 5800H (ORT fp32)	12.1	2.2	0.7
Jetson Nano (TRT fp32)	46.6	7.3	1.0

Table 8: **DepthStream vs. learned propagation** (VeloDepth numbers from their paper; both on 0.3–0.5 MP inputs).

	VeloDepth	DepthStream (ours)
propagation module	55 M, trained	none
training cost	8×4090, 6 days	—
base model	409 M (UniDepthV2)	6 M distilled
keyframe trigger	learned gating	residual (free)
hardware	RTX 3090	RTX 3060 Laptop 6GB
throughput	26.2 FPS	142–176 FPS

yet reproduced, so our comparison to it is architectural and latency-based rather than metric-exact.

5.2 When does amortization stop paying?

Three boundaries emerged from our experiments. *Occlusion density*: propagation error concentrates at disocclusions (Fig. 7); adaptive scheduling hides the symptom at scene level but a per-pixel mask with fallback inference is the natural next step. *Extreme intervals*: beyond $K{\approx}15$ the accumulation curve (Fig. 4) falls below the dense baseline, so the cadence should stay within the range where decay is flatter than the model's own frame-to-frame variance — empirically $K{\leq}10$ here. *Model mismatch*: if the runtime's student is *not* distilled from the referee model, a disagreement space re-opens and selective correction (Sec. 4.7) or learned propagation [3] becomes rational again.

6 Limitations

(i) Flow propagation fails at disocclusions and large rotations; the adaptive scheduler masks the symptom (early re-anchoring) but does not model occlusion explicitly. (ii) Our evaluation covers one synthetic and nine real sequences; extending Bonn to all published sequences and adding TUM would strengthen the real-data claim. (iii) Power consumption is not yet reported for the platform table, and GPU-side evaluation uses one consumer laptop GPU; Orin-class Jetson results are expected to transfer directly (same TensorRT pipeline, official DepthART scripts exist). (iv) The student consumes no camera intrinsics here; the metric variant of DepthART would require propagating K jointly with the field. (v) VeloDepth [3] reports a temporal-consistency metric (τ_5^{SSI}) that we have not

7 Conclusion

For a depth model distilled to its last parameter, the remaining efficiency is temporal — and it can be harvested for free. DepthStream runs a frozen 6 M student on keyframes, transports the anchored depth field with backward flow, and re-anchors fresh predictions with a closed-form affine fit; a photometric monitor guards the schedule. The free lunch is real and strict: at $K{=}3$ the stream matches per-frame accuracy exactly while cutting latency and raising temporally consistent pixels from 55% to 73%, on synthetic and real benchmarks alike. The lunch has charted boundaries — a cadence knee at $K^{*}{\approx}10$, occlusion-dense penalties for blending, and a distillation condition — and its purchasing power flips the deployment map: 90 FPS stable depth on a 2019 $99 board, 249 FPS on a CPU, identical accuracy on either backend. For practitioners the rule is one sentence: propagate — do not correct; and re-anchor — do not smooth.

Reproducibility and licensing

All tables are produced by single commands (`eval_sintel_v2.py`, `eval_bonn.py`, `bench_cpu.py`; figures by `plots.py`, `qual_figs.py`, and `draw_framework.py`). The student checkpoint and teacher weights are public; the student and parts of its supervision inherit CC BY-NC 4.0 terms from DepthART and Depth Anything V2, which restricts commercial use of the released weights (the runtime itself has no such restriction).

Sintel and Bonn are used under their respective research licenses.

References

[1] F. Xue et al. "DepthART: Scaling Foundation Monocular Depth to Tiny Models," *ACM Multimedia*, 2026. arXiv:2607.17099.

[2] L. Yang et al. "Depth Anything V2," *NeurIPS*, 2024. arXiv:2406.09414.

[3] L. Piccinelli et al. "Video Depth Propagation (VeloDepth)," *3DV*, 2026. arXiv:2512.10725.

[4] W. Wang et al. "Neural Video Depth Stabilizer," *ICCV*, 2023.

[5] W. Cao et al. "DepthCrafter: Generating Consistent Long Depth Sequences for Open-world Videos," *CVPR*, 2025.

[6] J. Shao et al. "Learning Temporally Consistent Video Depth from Video Diffusion Models" (ChronoDepth), *CVPR*, 2025.

[7] S. Chen et al. "Video Depth Anything: Consistent Depth Estimation for Super-Long Videos," *CVPR*, 2025. arXiv:2501.12375.

[8] J. Kopf et al. "Consistent Video Depth Estimation," *ACM TOG (SIGGRAPH)*, 2021.

[9] S. Li, Y. Luo, Y. Mao et al. "Enforcing Temporal Consistency in Video Depth Estimation" (TCMonoDepth), *ICCV Workshops*, 2021.

[10] R. Ranftl et al. "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," *IEEE TPAMI*, 2022.

[11] L. Yang et al. "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," *CVPR*, 2024. arXiv:2401.10891.

[12] S. F. Bhat et al. "ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth," *ICRA*, 2023.

[13] B. Ke et al. "Repurposing Diffusion-based Image Generators for Monocular Depth Estimation," *CVPR*, 2024.

[14] M. Gui et al. "DepthFM: Fast Monocular Depth Estimation with Flow Matching," *ACM Multimedia*, 2024.

[15] M. Pietruszka et al. "Real-Time Monocular Depth Estimation on Embedded Systems," *arXiv:2308.10569*, 2023.

[16] X. Zhu et al. "Deep Feature Flow for Video Recognition," *CVPR*, 2017.

[17] C. Cheng and M. Schiele. "XMem: Long-Term Video Object Segmentation with an Atkinson-Shiffrin Memory Model," *ECCV*, 2022.

[18] Y. Leviathan et al. "Fast Inference from Transformers via Speculative Decoding," *ICML*, 2023.

[19] P. Li, Y. Ding, L. Li, J. Guan, and Z. Li. "Towards Practical Consistent Video Depth Estimation," *ACM ICMR*, 2023.

[20] W. Yin et al. "Metric3D v2: Geometric Foundation Model," *IEEE TPAMI*, 2024.

[21] L. Piccinelli et al. "UniDepth: Universal Monocular Metric Depth Estimation," *CVPR*, 2024.

[22] D. Wofk et al. "FastDepth: Efficient Monocular Depth Estimation for Embedded Devices," *ICRA*, 2019.

[23] Z. Li et al. "PatchFusion: An End-to-End Tile-Based Framework for High-Resolution Monocular Metric Depth Estimation," *CVPR*, 2024.

[24] T. Kroeger et al. "Fast Optical Flow using Dense Inverse Search," *ECCV*, 2016.

[25] G. Farnebäck. "Two-Frame Motion Estimation Based on Polynomial Expansion," *SCIA*, 2003.

[26] D. J. Butler et al. "A Naturalistic Open Source Movie for Flow Evaluation," *ECCV*, 2012.

[27] E. Palazzolo et al. "ReFusion: 3D Reconstruction in Dynamic Environments using a Time-of-Flight Sensor" (Bonn RGB-D Dynamic), *IROS*, 2019.

[28] R. Ranftl et al. "Vision Transformers for Dense Prediction" (DPT), *ICCV*, 2021.

[29] D. Qin et al. "MobileNetV4 – Universal Models for the Mobile Ecosystem," *arXiv:2404.10518*, 2024.

[30] NVIDIA. "TensorRT," https://developer.nvidia.com/tensorrt.